Smart Moisture-Sensing Glove and Regional Waste Intelligence Platform

A Hardware and Software System for Faster Wet/Dry Waste Segregation and Waste-to-Energy Feedstock Planning

Team Project Report
Environmental Hacks – Waste and Energy Track

October 2026

Contents

1 Abstract **4**

2 Problem Statement and Motivation **4**

2.1 The problem 4

2.2 Why moisture matters for energy 4

2.3 Policy context 4

3 Objectives and Scope **4**

4 System Overview **5**

4.1 End-to-end data flow 5

5 Hardware Section **5**

5.1 Component list 6

5.2 What each sensor measures 6

5.3 Why a contact probe instead of hovering 7

5.4 Pin mapping (Arduino Uno) 7

5.5 Glove assembly 7

5.6 Power and safety 7

5.7 Known hardware limitations 7

6 What We Measure **8**

6.1 Raw measurements per reading 8

6.2 Derived quantities 8

6.3 Measurement protocol 8

7 Data Collection and Calibration **8**
7.1 Calibration . . . 8
7.2 Training dataset . . . 9
7.3 Train and test split . . . 9

8 Software Section **9**
8.1 Arduino firmware . . . 9
8.2 Gateway program . . . 9
8.3 Backend API . . . 10
8.4 Data model (DynamoDB) . . . 10
8.5 Dashboard . . . 10

9 Where and How AI Is Used **10**
9.1 1. Per-reading wet/dry classification (core) . . . 10
9.2 2. Moisture score estimation (core) . . . 11
9.3 3. Regional pattern recognition (core analytics) . . . 11
9.4 4. Generative insights (optional extension) . . . 11
9.5 Energy Suitability Index . . . 11

10 AWS Architecture and Deployment **12**
10.1 Services and their roles . . . 12
10.2 How the model is served . . . 12
10.3 Build It and Ship It . . . 12
10.4 Infrastructure as code . . . 12
10.5 Ingest function logic . . . 13
10.6 Deployment steps . . . 13
10.7 Cost and security . . . 13

11 How Everything Works Together: A Walk-Through **13**

12 Implementation Plan **14**

13 Evaluation and Demo Plan **14**
13.1 Evaluation . . . 14
13.2 Three-minute demo outline . . . 14

14 Limitations and Future Work **14**
14.1 Limitations . . . 14
14.2 Future work . . . 14

15 Conclusion **15**

1 Abstract

Waste workers in India separate wet and dry waste largely by hand, which is slow, tiring and unhygienic. At the same time, waste-to-energy (WtE) and refuse-derived-fuel (RDF) plants need feedstock with low moisture content, but municipalities rarely know which areas produce such waste. This project proposes a low-cost wearable glove with a contact moisture probe, a temperature and humidity sensor and an optional gas sensor. The glove gives a worker an instant wet/dry indication and records every reading with a location tag. The readings are sent to a cloud platform built on Amazon Web Services (AWS), where machine learning models classify the waste and aggregate the data per colony or ward. The result is a map and a ranking of areas by how suitable their waste is for energy recovery, which helps planners route dry waste directly to energy plants and target awareness campaigns where segregation is poor.

2 Problem Statement and Motivation

2.1 The problem

- Wet (biodegradable) waste forms a large share of Indian municipal solid waste, commonly quoted at about 50–55%.
- When wet and dry waste are mixed, the dry fraction becomes damp and loses value for recycling and for energy recovery.
- Workers at collection points and transfer stations sort mixed waste by sight, smell and touch. This is repetitive, exposes them to health risks and gives no data to anyone.
- Planners have no reliable, area-wise data on how wet or dry the waste of each colony is.

2.2 Why moisture matters for energy

WtE plants depend on the calorific value and moisture content of their feedstock. Studies of Indian municipal waste report average moisture of roughly 41–52% and calorific values often below what efficient combustion needs, and a mismatch between plant requirements and incoming waste is a common cause of plant failure. RDF grades are also defined with maximum moisture limits. Moisture is therefore the single most practical property to measure and track.

2.3 Policy context

The Solid Waste Management Rules, 2026 (in force from 1 April 2026) require segregation at source into four streams: wet, dry, sanitary and special-care waste, and introduce online tracking of waste handling. A system that measures segregation quality and produces area-wise data directly supports this framework.

3 Objectives and Scope

1. Build a wearable glove that gives a worker an immediate wet/dry/uncertain indication when touching waste.
2. Record each measurement with time, location and area identifier.
3. Train a machine learning classifier on labelled readings and report honest accuracy.
4. Estimate a relative *moisture score* for each item and each load.
5. Aggregate data per colony or ward and compute an *Energy Suitability Index* (ESI).
6. Present the results on a web dashboard and deploy the backend on AWS.

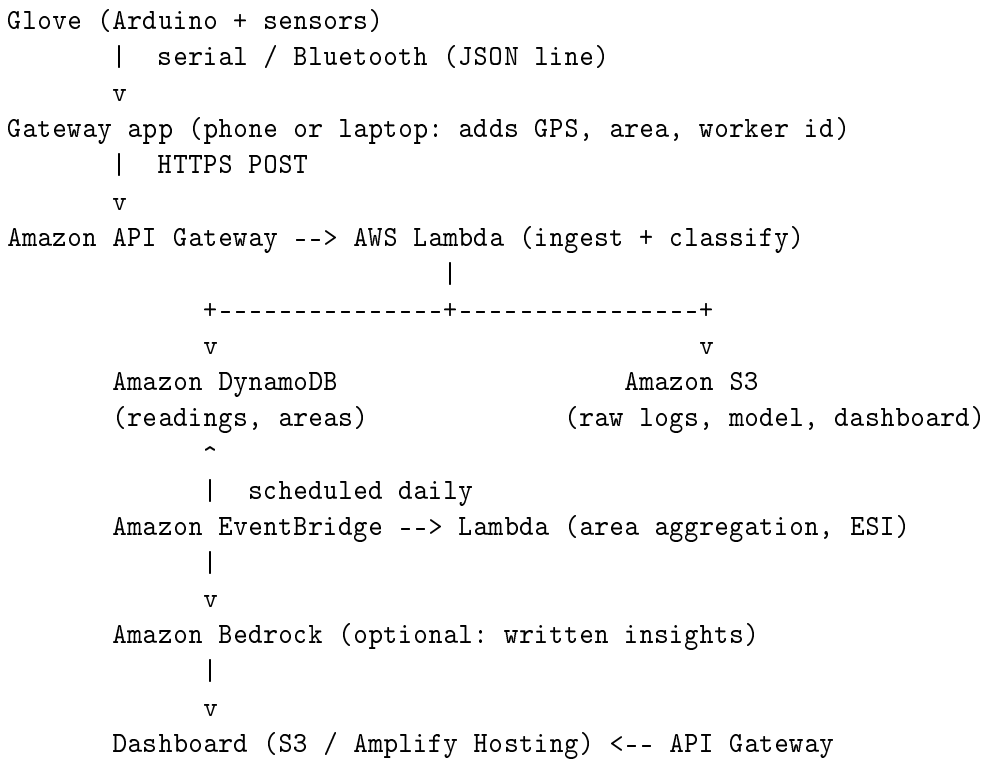
Scope note. The system measures electrical conductivity of the contact surface, air temperature and air humidity. It does *not* measure calorific value. The moisture score is a relative indicator, which can later be calibrated against laboratory moisture tests.

4 System Overview

The system has four layers.

Layer	Role
Sensing layer	Glove with probe and sensors on an Arduino; takes readings and shows an instant result with LED and buzzer.
Gateway layer	A phone or laptop program that receives readings from the Arduino, attaches GPS location and area, and uploads to the cloud.
Cloud layer	AWS services that store data, run the machine learning model, aggregate by area and serve the API.
Presentation layer	Web dashboard with a map, area rankings, trends and recommendations.

4.1 End-to-end data flow



5 Hardware Section

This section describes only the physical device. Software and AI are covered in later sections.

5.1 Component list

Component	Quantity	Purpose
Arduino Uno or Nano	1	Reads sensors, runs simple on-device logic, sends data.
LM393 moisture comparator board	1	Conditions the signal from the contact probe; analog output is used.
Contact probe (two stainless electrodes)	1	Touches the waste; resistance between electrodes depends on moisture.
DHT11 or DHT22 sensor	1	Measures ambient air temperature and relative humidity as a reference.
Gas sensor (MQ-135 or similar), optional	1	Indicates volatile compounds from decomposing organic waste.
IR temperature sensor (MLX90614), optional	1	Non-contact surface temperature, if available.
Green and red LED	1 each	Visual result: wet or dry.
Buzzer	1	Audible result for noisy environments.
Push button	1	Starts a measurement.
Bluetooth module (HC-05) or USB cable	1	Link to the gateway.
Battery pack (power bank or 9 V)	1	Portable power.
Resistors, jumper wires, small breadboard or perfboard	as needed	Wiring.
Work glove (nitrile-coated) and waterproof pouch	1	Mounting and protection.

5.2 What each sensor measures

Sensor	Quantity measured	Output	Role in system
Contact probe + LM393	Electrical conductivity of the touched surface (moisture proxy)	Analog value, 0–1023	Primary wet/dry feature
DHT sensor	Air temperature and relative humidity	Degrees Celsius, percent	Reference for drift and weather correction
Gas sensor	Volatile gas concentration (relative)	Analog value	Detects decomposing organic matter
IR sensor	Surface temperature	Degrees Celsius	Decomposition heat indicator

5.3 Why a contact probe instead of hovering

A humidity sensor held above a pile mostly reports the humidity of the surrounding air, responds slowly and cannot distinguish individual items. A probe that touches the material measures the material itself, so the primary decision is based on contact and the humidity sensor is kept as a reference input.

5.4 Pin mapping (Arduino Uno)

Device pin	Arduino pin	Note
Moisture board VCC	D7	Powered only while reading to reduce electrode corrosion
Moisture board GND	GND	
Moisture board AO	A0	Analog moisture value
DHT data	D2	
Gas sensor AO	A1	Optional; needs warm-up time
Green LED (with resistor)	D8	Dry indication
Red LED (with resistor)	D9	Wet indication
Buzzer	D10	
Push button	D3	Start measurement, using internal pull-up
Bluetooth TX/RX	D4/D5	Software serial; optional if USB is used

5.5 Glove assembly

1. Fix the two stainless electrodes on a non-conductive strip, about 0.5–1 cm apart, and mount the strip on the index fingertip or on a short probe tip held by the glove.
2. Mount the Arduino and moisture board in a small waterproof pouch on the back of the hand or the forearm.
3. Keep the DHT sensor in a ventilated opening of the pouch so it samples air and not the pouch interior.
4. Route wires along the finger with flexible tape; keep all electronics away from the palm.
5. Place the LEDs and buzzer on the wrist where the worker can see and hear them.

5.6 Power and safety

- Operating voltage is 5 V from a small power bank; no mains connection is used on the glove.
- The probe runs at low voltage and low current, and is switched off between readings.
- Electrodes should be wiped or replaced between uses, and the glove should be cleaned as per local hygiene practice.
- Sharps, medical and hazardous waste must still be handled with normal protective equipment; the device does not replace it.

5.7 Known hardware limitations

- Metal items (cans, foil) can short the electrodes and read falsely as wet.
- Wet plastic with only a thin water film may read as dry or uncertain.
- Electrodes degrade with time; stainless steel and pulsed power reduce this.
- Readings drift with temperature, so the DHT reference is logged with every sample.

6 What We Measure

6.1 Raw measurements per reading

Field	Source	Description
`moisture_raw`	Probe, A0	Median of several ADC samples taken during contact.
`moisture_slope`	Probe, A0	Rate of change in the first second of contact.
`air_temp_c`	DHT	Ambient temperature.
`air_rh_pct`	DHT	Ambient relative humidity.
`gas_raw`	Gas sensor	Relative volatile gas level, if fitted.
`surface_temp_c`	IR sensor	Surface temperature, if fitted.
`timestamp`	Gateway	UTC time of reading.
`lat, lon`	Gateway (phone GPS)	Location of the reading.
`area_id`	Gateway	Colony, ward or collection point chosen by the worker.
`device_id`	Firmware	Identifier of the glove.
`label`	Training only	True class set by a person during data collection.

6.2 Derived quantities

- **Moisture score** $M \in [0, 1]$: a normalised value where 0 is completely dry and 1 is fully wet, produced by calibration or by the regression model.
- **Class**: *wet*, *dry* or *uncertain*.
- **Dry fraction** of an area: share of readings classified dry within a time window.
- **Energy Suitability Index (ESI)** of an area, defined in Section 9.5.

6.3 Measurement protocol

1. Worker presses the button; the Arduino powers the probe.
2. After a 50 ms settling delay, it takes 5 to 10 analog samples over about one second while the probe touches the item.
3. It computes the median and the slope, reads the DHT (and optional sensors), and powers the probe off.
4. It sends one JSON line to the gateway and shows the on-device result with LEDs and buzzer.
5. The gateway adds location and area and uploads the reading.

7 Data Collection and Calibration

7.1 Calibration

Before the model is trained, the probe is calibrated on reference materials: open air, a wet sponge, fresh vegetable scraps, dry paper, dry plastic and dry cloth. This gives the working range of `moisture_raw` and shows the direction of change (on many boards a higher raw value means drier). A first threshold is set from these readings and is used by the on-device logic.

7.2 Training dataset

We collect at least 100 to 200 labelled readings, covering:

- Wet class: vegetable and fruit peels, cooked food, tea leaves, wet paper or cardboard.
- Dry class: dry paper, dry cardboard, plastic packaging, dry cloth, dry wood.
- Hard cases: metal cans, wet plastic bottles, damp cloth, food-stained wrappers.

Each item is measured several times at different spots and on different days and temperatures. Labels are assigned by a person before the reading.

7.3 Train and test split

Readings are split by *session* (a session is one sitting of data collection) rather than randomly, so that the test set resembles new conditions. About 70% of sessions are used for training and 30% for testing.

8 Software Section

8.1 Arduino firmware

Responsibilities: read the button, pulse the probe, sample, compute the median and slope, read the DHT, apply the on-device threshold, drive LEDs and buzzer, and print one JSON line over serial.

```
// Simplified firmware outline
const int PWR = 7, AOUT = A0;
int readMedian(int n) {
  int v[10];
  for (int i = 0; i < n; i++) { v[i] = analogRead(AOUT); delay(100); }
  // sort v and return the middle value
}
void loop() {
  if (digitalRead(3) == LOW) {          // button pressed
    digitalWrite(PWR, HIGH); delay(50); // power the probe
    int m = readMedian(7);
    digitalWrite(PWR, LOW);
    // read DHT, compute class from threshold
    // Serial.println("{...json...}");
  }
}
```

8.2 Gateway program

A short Python program (or a simple phone app) runs on the worker's phone or on a laptop at the demo. It reads the JSON line from the Arduino, adds the timestamp, GPS and area, and sends it by HTTPS to the API. If there is no network, it stores readings in a local file and uploads them later, so work in low-connectivity areas is not lost.

8.3 Backend API

Method	Path	Purpose
POST	`/readings`	Receive a reading, classify it, store it, return the class.
GET	`/areas`	List areas with dry fraction and ESI.
GET	`/areas/{id}`	Details and trend of one area.
GET	`/insights`	Written recommendations (optional).

Example request body:

```
{
  "device_id": "glove-01",
  "timestamp": "2026-10-10T09:30:12Z",
  "moisture_raw": 312,
  "moisture_slope": -41,
  "air_temp_c": 29.4,
  "air_rh_pct": 61.0,
  "gas_raw": 188,
  "lat": 28.7501, "lon": 77.1177,
  "area_id": "ward-12-colony-b"
}
```

8.4 Data model (DynamoDB)

Table	Keys	Main attributes
`Readings`	partition: `area_id`, sort: `timestamp`	all raw fields, predicted class, moisture score
`AreaStats`	partition: `area_id`, sort: `date`	count, dry fraction, mean moisture score, ESI, cluster label

8.5 Dashboard

A static web page hosted on AWS and calling the API shows:

- A map with a marker or heat layer per area coloured by dry fraction (the final interface may use any palette; this report uses none).
- A ranked table of areas by ESI.
- A time trend of moisture score for the selected area.
- A recommendations panel (for example, which areas to schedule for dry-waste pickup to the plant).
- A live feed of the latest readings with their classes, used in the demo.

9 Where and How AI Is Used

AI is used in four places. The first two are core to the project; the last two are extensions to be included only if time allows.

9.1 1. Per-reading wet/dry classification (core)

What. A supervised classifier predicts *wet*, *dry* or *uncertain* for each reading.

Inputs. `moisture_raw`, `moisture_slope`, air temperature, air humidity, the difference between probe reading and ambient humidity, and the gas and IR values if available.

Models. We start with a threshold baseline, then compare logistic regression, a decision tree and a random forest (or gradient boosting) using scikit-learn. The simplest model that performs well is chosen.

Uncertain class. If the predicted probability is below a confidence limit, the reading is marked uncertain and the worker is told to retry or sort by eye. This is safer than forcing a wrong answer.

Evaluation. Accuracy, precision, recall, F1 score and a confusion matrix on the held-out sessions. We report the hard cases (metal, wet plastic) separately.

Where it runs. The model is trained offline on a laptop. A compact version (threshold rules or a small decision tree exported to plain code) runs on the Arduino for instant feedback without internet. The full model runs in AWS Lambda for the stored result and for analytics.

9.2 2. Moisture score estimation (core)

A regression model (or the calibration curve) converts the raw readings into a moisture score M between 0 and 1. If a kitchen scale and oven are available, a small number of samples can be weighed wet and dry to obtain true moisture percentage and calibrate M against it. If not, M remains a relative score, and the report states this clearly.

9.3 3. Regional pattern recognition (core analytics)

- **Aggregation.** Per area and per day: number of readings, dry fraction, mean moisture score.
- **Clustering.** K-means or hierarchical clustering groups areas into profiles such as "high-dry, low-moisture" or "mostly wet".
- **Trend detection.** Rolling averages show whether segregation in an area is improving.
- **Anomaly detection.** An Isolation Forest flags unusual loads, for example a sudden jump in moisture or gas, which may indicate contamination.

9.4 4. Generative insights (optional extension)

Amazon Bedrock (or the Strands Agents SDK) turns the area statistics into a short written recommendation for a municipal officer, such as which colonies should be prioritised for dry-waste collection this week and where awareness drives are needed. The language model never decides the classification; it only explains numbers already computed, which keeps it reliable.

9.5 Energy Suitability Index

For each area and period we compute

$$\mathrm{ESI} = 100 \times \big(w_1 \, (1 - \bar{M}) + w_2 \, f_{\mathrm{dry}} + w_3 \, (1 - f_{\mathrm{unc}}) \big) \tag{1}$$

where $\bar{M}$ is the mean moisture score, f_{dry} is the dry fraction, f_{unc} is the share of uncertain readings, and the weights satisfy $w_1 + w_2 + w_3 = 1$ (initially 0.5, 0.3 and 0.2). ESI is a planning aid, not a measurement of calorific value, and the weights can be tuned with plant feedback.

10 AWS Architecture and Deployment

10.1 Services and their roles

AWS service	Role in this project
Amazon API Gateway	Public HTTPS endpoints for the gateway app and dashboard.
AWS Lambda	Ingest function (validate, classify, store) and daily aggregation function.
Amazon DynamoDB	Stores readings and area statistics.
Amazon S3	Stores raw logs, the trained model file and the static dashboard.
AWS Amplify Hosting or S3 website hosting	Serves the dashboard.
Amazon EventBridge	Triggers the daily aggregation Lambda on a schedule.
Amazon CloudWatch	Logs and basic monitoring of Lambda and API errors.
Amazon Cognito (optional)	Login for workers and officers.
Amazon SNS (optional)	Alerts when an anomaly is detected.
Amazon SageMaker (optional)	Managed training and hosting, if time and credits allow.
Amazon Bedrock (optional)	Written insights for officers.

10.2 How the model is served

To keep deployment simple and cheap, the classifier is trained on a laptop and exported as a small file (for example a JSON of decision-tree rules or an ONNX model). The ingest Lambda loads this file from S3 once at start-up and applies it to each reading. This avoids heavy libraries inside Lambda. If a larger model is later needed, it can be moved to a SageMaker endpoint without changing the API.

10.3 Build It and Ship It

The hackathon allows two ways to qualify. We plan for both:

- **Build It (local):** run the API and functions locally using AWS SAM CLI and LocalStack, with no AWS account needed.
- **Ship It (deployed):** deploy the same SAM template to a real AWS account and submit the live URL.

10.4 Infrastructure as code

All resources are described in one AWS SAM template so that deployment is repeatable.

```
# template.yaml (outline)
Resources:
  ReadingsTable:   AWS::DynamoDB::Table
  AreaStatsTable:  AWS::DynamoDB::Table
  ModelBucket:     AWS::S3::Bucket
  IngestFunction:  AWS::Serverless::Function   # POST /readings
  AreasFunction:   AWS::Serverless::Function   # GET /areas
```

```
  AggregateFunction: AWS::Serverless::Function # EventBridge schedule
  Api:               AWS::Serverless::HttpApi
```

10.5 Ingest function logic

```
def handler(event, context):
    r = parse_and_validate(event["body"])
    features = build_features(r)
    cls, prob = model.predict(features)      # loaded from S3
    if prob < CONFIDENCE: cls = "uncertain"
    r["class"], r["moisture_score"] = cls, score(features)
    table.put_item(Item=r)                   # DynamoDB
    return {"statusCode": 200, "body": json.dumps({"class": cls})}
```

10.6 Deployment steps

1. Create an AWS account on the Free Tier and apply the hackathon credits if needed.
2. Install the AWS CLI and SAM CLI and configure credentials.
3. Train the model locally and upload the exported model file to the S3 bucket.
4. Run `sam build` and `sam deploy -guided`; note the API URL.
5. Upload the dashboard files to S3 website hosting or Amplify, with the API URL set in its configuration.
6. Point the gateway program at the API URL and test with a real glove reading.
7. Check CloudWatch logs, fix errors and record the demo.

10.7 Cost and security

All services used are covered by the Free Tier or small credits at demo scale. API keys or Cognito protect the write endpoint, IAM roles give each Lambda only the access it needs, and no personal data is stored: readings carry a device identifier and an area, not a person's name.

11 How Everything Works Together: A Walk-Through

1. A worker at a collection point selects the area on the gateway app and puts on the glove.
2. The worker touches an item and presses the button. The probe is powered for about one second and the readings are taken.
3. The Arduino immediately lights green (dry) or red (wet), or beeps twice for uncertain, using its on-device rule. The worker places the item accordingly.
4. The Arduino sends the reading to the gateway, which adds GPS, time and area and posts it to API Gateway.
5. The ingest Lambda runs the full model, stores the reading in DynamoDB and returns the class.
6. Every night, EventBridge starts the aggregation Lambda. It computes dry fraction, mean moisture score, ESI, clusters and anomalies for each area, and writes them to DynamoDB.
7. The dashboard reads the area statistics. An officer sees which colonies have the best energy-suitable waste and where segregation is poor.
8. Dry, low-moisture loads from the best areas are scheduled for direct transport to the energy plant, and campaigns are planned for the poorest areas.

12 Implementation Plan

Stage	Tasks	Owner
1. Hardware test	Wire probe, DHT, LEDs; calibrate; confirm wet and dry readings separate.	Hardware member
2. Data collection	Collect labelled readings across items, sessions and conditions.	Whole team
3. Model	Train baseline and classifiers; evaluate; export model.	AI member
4. Cloud	SAM template, Lambda functions, DynamoDB, S3, API; deploy.	Cloud member
5. Gateway and dashboard	Gateway script, map and ranking page.	Software member
6. Integration and demo	End-to-end test, three-minute video, write-up.	Whole team

13 Evaluation and Demo Plan

13.1 Evaluation

- Classification accuracy, precision, recall, F1 and confusion matrix on unseen sessions.
- Comparison of the glove against manual sorting for speed on a small mixed pile.
- Latency from button press to dashboard update.
- Honest listing of failure cases (metal, wet plastic).

13.2 Three-minute demo outline

1. Problem and who is affected (30 seconds).
2. Glove touching wet and dry items with instant LED result (45 seconds).
3. Reading appearing on the dashboard and in DynamoDB (30 seconds).
4. Area map and ESI ranking, with a recommendation (45 seconds).
5. Architecture on AWS and results of the evaluation (30 seconds).

14 Limitations and Future Work

14.1 Limitations

- The probe gives a relative moisture indication, not calorific value or exact moisture percentage.
- Performance on materials outside the training set is unknown until more data is collected.
- The prototype separates wet from dry only; the 2026 rules also require sanitary and special-care waste to be kept apart.
- A small, single-city dataset limits how far results generalise.

14.2 Future work

- Replace the Arduino with an ESP32 for direct WiFi upload.
- Add a camera model to combine visual material recognition with moisture sensing.
- Calibrate the moisture score with laboratory tests and plant feedback.

- Extend classes to the four streams of the 2026 rules.
- Integrate with municipal tracking portals and truck routing.
- Use the data to estimate regional RDF and biogas potential.

15 Conclusion

The project turns a manual and data-less task into a measured one. A cheap glove gives workers instant feedback, a cloud platform on AWS stores and classifies every reading, and machine learning converts the readings into area-level intelligence that helps send the right waste to the right plant. The design is deliberately modest in its claims: it measures moisture by contact, labels data honestly, and reports its limits, while leaving a clear path to a field-ready system.

References

1. Ministry of Environment, Forest and Climate Change, Government of India. Solid Waste Management Rules, 2026 (notified 27 January 2026, in force from 1 April 2026).
2. Central Pollution Control Board and NEERI study of municipal solid waste in 59 Indian cities (2004–05), as cited in public analyses of waste-to-energy feedstock quality.
3. Guidelines on usage of refuse-derived fuel in various industries, Government of India.
4. WeMakeDevs and AWS, Bharat Builds Tour, Environmental Hacks (Waste and Energy track), event rules and judging criteria.